Using Promise

Callback hell typically occurs when dealing with *a series of asynchronous operations*, such as making API requests, where one operation depends on the result of another or previous one. The callback itself has to call functions that accept a callback. As a result, they are nested.

Promise
A promise represents the eventual completion or failure of an asynchronous operation, allows you *to chain operations* using methods like then() and catch(). Promise simplifies error handling, eliminates nested callback functions, and improves code readability.

Create a Promise ---
A promise represents the eventual completion or failure of an asynchronous operation

```
function fetchData(URL) {
  return  new Promise(function(resolve, reject) { // executor function
    // asynchronous operation produces result or error
    …
  // let success = true; // or false
    // let result/error = …
    if (success) {
        resolve(result);
    } else {
        reject(error);
  }
  });
}
```

Consume a Promise ---
Once you have a reference, you can attach callback function by using the .then and .catch methods.
Once the promise is resolved, the .then callback method will be called with the resolved value.
And if the promise is rejected, the .catch method will be called with an error message.

```
fetchData(URL)
.then(function(result) { // callback function // This runs if the promise is fulfilled
  console.log(result);
})
.catch(function(error) { // This runs if the promise is rejected
  console.log(error);
});
```

As an example, here's how you would get a JSON resource using the Fetch API, and parse it, using promises:

```
const getFirstUserData = () => {
  // get users list
  return (
    fetch('/users.json')
    .then((response) => response.json())   // parse JSON
    .then((users) => users[0]                    // pick first user
    .then( (user) => fetch(`/users/${user.name}`) )    // get user data
    .then((userResponse) => userResponse.json())   // parse JSON
  )
}

getFirstUserData()
```

And here is the same functionality provided **using await/async**:

```
const getFirstUserData = async () => {
  // get users list
  const response = await fetch('/users.json')
  const users = await response.json()                // parse JSON
  const user = users[0]                                    // pick first user
  const userResponse = await fetch(`/users/${user.name}`)       // get user data
  const userData = await userResponse.json()                        // parse JSON

  return userData
}
```